

R Programming Concepts Used in FitTrack AI Application

Academic Documentation & Viva Preparation Guide

A comprehensive guide explaining the backend logic, mathematical computations, and API architecture built using R.

Table of Contents

1. [Introduction to R Programming](#)
 2. [List of R Programming Topics Used](#)
 3. [Detailed Explanation of Each Topic](#)
 4. [R Code Snippets from Project](#)
 5. [Role of R in the System](#)
 6. [Advantages of Using R in This Project](#)
 7. [Comparison: R vs Python](#)
 8. [Conclusion](#)
-

1. Introduction to R Programming

What is R programming?

R is a powerful, open-source programming language widely used for statistical computing, data analysis, and mathematical modeling. Originally designed for statisticians, it has grown extensively to support backend logic, data processing, and API development in modern applications.

Key features of R:

- **Open-source:** Free to use and backed by a large developer community.
- **Rich Ecosystem:** Offers thousands of packages (like `plumber` for APIs and `DBI` for database interaction).
- **Vectorized Operations:** Can perform calculations on entire arrays/vectors simultaneously without complex loops.
- **Platform Independent:** Runs seamlessly on Windows, macOS, and Linux.

Why R is suitable for this project:

In the **FitTrack AI** application, R is not used to build the user interface, but rather to serve as the **core calculation engine**. Fitness applications require precise mathematical formulas (BMI, BMR, Calorie estimation). R executes these formulas dependably and exposes them as a backend API that the Flutter frontend can consume safely.

2. List of R Programming Topics Used

To build the FitTrack AI backend, several core R concepts were implemented. These include:

1. **Variables and Data Types**
 2. **Arithmetic Operations**
 3. **Conditional Statements (`if-else`)**
 4. **Functions**
 5. **Mathematical Calculations**
 6. **Data Handling (Database Connectivity)**
 7. **API Creation using Plumber**
-

3. Detailed Explanation of Each Topic

3.1 Variables and Data Types

- **Definition:** Variables act as containers for storing data values. In R, data types don't need to be declared strictly; R infers them. Important types used include *Numeric* (for weights, heights), *Character* (for names, genders), and *Logical* (TRUE/FALSE).

- **Syntax:** `variable_name <- value` or `variable_name = value`
- **Example Code:**

```
weight_kg <- 70.5      # Numeric
gender <- "Male"       # Character
is_active <- TRUE      # Logical
```

- **Usage in the Project:** User inputs from the frontend (like age, weight, and height) are assigned to R variables for processing.
- **Why It Was Used:** To temporarily hold user stats in memory during API requests.
- **Benefits: Simplicity**—R's dynamic typing means less boilerplate code.

3.2 Arithmetic Operations

- **Definition:** Mathematical operations such as addition, subtraction, multiplication, division, and exponentiation.
- **Syntax:** `+`, `-`, `*`, `/`, `^`
- **Example Code:**

```
height_m <- height_cm / 100
bmi <- weight_kg / (height_m ^ 2)
```

- **Usage in the Project:** Used extensively to compute Body Mass Index (BMI).
- **Why It Was Used:** BMI requires division and power variables, which R handles efficiently.
- **Benefits: Accuracy** in floating-point operations makes health metrics reliable.

3.3 Conditional Statements (`if-else`)

- **Definition:** Decision-making structures that execute specific blocks of code based on a condition evaluating to TRUE or FALSE.
- **Syntax:**

```
if (condition) {
  # execute if TRUE
} else {
  # execute if FALSE
}
```

- **Example Code:**

```
if (goal == "Lose weight") {  
  calories <- bmr - 500  
} else {  
  calories <- bmr  
}
```

- **Usage in the Project:** To apply different BMR formulas for Males vs Females, and to adjust daily calorie recommendations based on user goals (Lose, Gain, Maintain).
- **Why It Was Used:** User physiology and goals necessitate variable outcomes.
- **Benefits:** **Accuracy** in personalizing fitness regimens.

3.4 Functions

- **Definition:** A reusable block of code that performs a specific task. R allows creating custom functions to encapsulate logic.
- **Syntax:**

```
function_name <- function(arg1, arg2) {  
  # body of function  
  return(result)  
}
```

- **Example Code:**

```
get_db_connection <- function() {  
  con <- dbConnect(...)  
  return(con)  
}
```

- **Usage in the Project:** We created modular blocks (e.g., fetching a DB connection, API endpoint functions).
- **Why It Was Used:** To prevent repetitive code and keep the script clean.
- **Benefits:** Improves **Simplicity** and **Maintainability**.

3.5 Mathematical Calculations

- **Definition:** Utilizing built-in functions designed for numerical data transformation.
- **Syntax & Example:** `round(3.14159, 2)` outputs `3.14`

- **Usage in the Project:** Built-in core functions like `round()`, `as.numeric()`, and custom formulas (Mifflin-St Jeor Equation).
- **Why It Was Used:** We need integer or 2-decimal outputs for database storage rather than infinitely tall decimals.
- **Benefits: Performance** (built-in functions are highly optimized in C) and **Accuracy**.

3.6 Data Handling (DBI and RMariaDB)

- **Definition:** Managing permanent records inside a SQL database programmatically via R.
- **Syntax:** `dbGetQuery(connection, "SQL Query")`
- **Example Code:**

```
user_data <- dbGetQuery(con, "SELECT * FROM users WHERE id = ?", params = list(user_id)
```

- **Usage in the Project:** Linking R to the MySQL/MariaDB `fittrack_ai` database to store and retrieve user profiles, and attendance logs securely.
- **Why It Was Used:** To preserve user data between app sessions.
- **Benefits:** Promotes **Scalability**. R can handle vast datasets and multiple queries effectively.

3.7 API Creation using Plumber

- **Definition:** `Plumber` is a prominent R package that converts regular R code into a web API by using special decorator comments (`#*`).
- **Syntax:**

```
#* @get /endpoint
function() { ... }
```

- **Example Code:**

```
#* @get /user
function(user_id, res) {
  # Fetch logic here
}
```

- **Usage in the Project:** All the R algorithms are exposed as RESTful API paths (e.g., `GET /user`, `POST /user`, `POST /attendance`).

- **Why It Was Used:** The frontend relies on HTTP API responses. Plumber is the easiest bridge from pure R math logic to standard web APIs.
 - **Benefits:** Exceptional **Simplicity** when hooking R backends to modern mobile frontends.
-

4. R Code Snippets from Project

Below are the exact R algorithms used in FitTrack AI, alongside line-by-line explanations for Viva readiness.

4.1 BMI Calculation

Code:

```
height_cm <- as.numeric(height_cm)
weight_kg <- as.numeric(weight_kg)
bmi <- weight_kg / ((height_cm / 100) ^ 2)
bmi <- round(bmi, 2)
```

Explanation:

- `as.numeric()` : Ensures inputs received as text are converted to numbers.
- `height_cm / 100` : Converts centimeters to meters.
- `^ 2` : Squares the height in meters.
- `weight_kg / ...` : Core BMI formula calculation.
- `round(bmi, 2)` : Rounds the result to two decimal places. **Input Output:** Takes string inputs of weight and height; returns a numerical BMI score (e.g., `22.5`).

4.2 BMR Calculation (Mifflin-St Jeor Equation)

Code:

```
if (gender == "Female") {
  bmr <- 10 * weight_kg + 6.25 * height_cm - 5 * age - 161
} else {
  bmr <- 10 * weight_kg + 6.25 * height_cm - 5 * age + 5
}
bmr <- round(bmr, 2)
```

Explanation:

- Checks string condition: `if (gender == "Female")`
- If female: applies specific negative constant (`- 161`).
- Else (Male/Other): applies positive constant (`+ 5`).
- Variables `weight_kg`, `height_cm`, and `age` are instantly multiplied by the standard metabolic constants. **Input Output:** Takes gender, weight, height, and age; outputs the Basal Metabolic Rate (BMR)—the calories bodies burn while resting.

4.3 Calorie Logic (Bulk/Cut/Maintain)

Code:

```
if (goal == "Lose weight") {  
  calories <- bmr - 500  
} else if (goal == "Gain muscle") {  
  calories <- bmr + 300  
} else {  
  calories <- bmr  
}  
calories <- round(calories)  
protein_g <- round(2 * weight_kg)
```

Explanation:

- Evaluates the `goal` variable supplied by the frontend interface.
- **Cut:** Subtracts 500 calories for fat loss.
- **Bulk:** Adds 300 calories for muscle weight gain.
- **Maintain:** Provides the raw BMR metric.
- **Protein:** Uses `2 * weight_kg` to calculate a standard protein safety margin of 2g per Kg of body weight. **Input Output:** Takes `bmr` and `goal` ; outputs final personalized `calories` and `protein_g` per day.

4.4 API Endpoint Creation using Plumber

Code:

```
## Get user by ID
## @param user_id The user ID
## @get /user
function(user_id, res) {
  con <- get_db_connection()
  on.exit(dbDisconnect(con))
  query <- "SELECT * FROM users WHERE id = ?"
  user_data <- dbGetQuery(con, query, params = list(user_id))
  return(user_data[1, ])
}
```

Explanation:

- `## @param` : Plumber documentation identifying required inputs.
- `## @get /user` : Declares that making an HTTP GET request to `/user` triggers this function.
- `on.exit(dbDisconnect(con))` : A crucial safety feature in R that ensures the database connection closes safely even if an error crashes the script.
- `return(...)` : Sends the first row of user data back to the mobile app in JSON format automatically.

5. Role of R in the System

R acts as the central **Decision-Making Brain** and **Data Broker** of the application.

1. **Backend Logic:** R takes the raw data off of the user interface and transforms it using established health algorithms.
 2. **Processing User Data:** R bridges the gap between raw parameters (height, weight) and functional metrics (BMR, macros).
 3. **Frontend Communication:** R exposes its services via standard JSON responses over HTTP ports, meaning the Flutter application never has to understand R—it simply asks R for an answer, and R serves it instantly.
-

6. Advantages of Using R in This Project

- **Fast calculations:** R handles complex algebraic structures natively without demanding much system memory.
 - **Statistical accuracy:** Built for mathematicians, floating point errors are minimized compared to Javascript frameworks.
 - **Easy implementation of formulas:** Writing formulas in R looks identical to writing them in a standard algebra textbook.
 - **Scalability for AI:** By using R, future advanced Machine Learning algorithms (like predicting user weight loss trends using `keras` or `caret` libraries) can be integrated effortlessly without changing the tech stack.
-

7. Comparison: R vs Python for This Project

While Python is also excellent for backend data and logic, R was deliberately chosen for FitTrack AI due to:

- **Direct Formula Modeling:** R syntax treats mathematical definitions more naturally.
 - **Plumber API Simplicity:** R's `plumber` requires zero boilerplate setup to spin up an API (just `##` comments), whereas Python's FastAPI/Flask requires structuring router models, Pydantic schemas, and server app objects.
 - **Statistical Focus:** R excels at quick quantitative analytics, which align perfectly with the need to quickly crunch BMI/BMR/Meal combinations.
-

8. Conclusion

Summary: The FitTrack AI project leverages the mathematical prowess of R to handle its central fitness calculations securely and efficiently. By wrapping R logic with the `Plumber` library, it seamlessly transitions from a statistical calculator to a modern web backend that interfaces cleanly with databases (`RMariaDB`) and mobile clients.

Final Importance: R ensures that the "AI and tracking" capabilities of FitTrack are built on an incredibly solid foundation of accuracy. Utilizing R limits bugs related to mathematical calculation, isolates business

logic from the interface, and provides an instantly scalable framework should heavy machine learning be required in future versions.

Developed for academic evaluation. Best of luck in the Viva!